

# SIMATS ENGINEERING

## ASSESSMENT TOOL 2

R.HARIKA(192  
)

**COURSE NAME:** Database Management Systems  
**COURSE CODE:** CSA0501

### COURSE OUTCOME COVERED

**CO3:** *Analyze relational databases using functional dependencies, normalization (1NF to 5NF), and key constraints to identify redundancy and ensure data integrity. (BL2, BL3)*

Activity Tool : Experiments (High)  
Assessment Tool Weightage: 30%

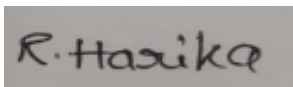
| QUESTIONS |                                                                                                                                                         |                  |       | Total Marks: 50 |
|-----------|---------------------------------------------------------------------------------------------------------------------------------------------------------|------------------|-------|-----------------|
| Q.No      | Question                                                                                                                                                | Bloom's Level    | Marks |                 |
| 1         | Analyze the case: An online shopping platform stores Order(OrderID, CustID, CustName, ProdID, ProdName, Qty, Price). Identify FDs and normalize to 3NF. | BL2 – Understand | 5     |                 |
| 2         | A hospital maintains Patient(PID, PName, DocID, DocName, Specialization, WardNo, WardName). Identify anomalies and normalize to BCNF.                   | BL3 – Apply      | 5     |                 |
| 3         | Case study: A university stores Enrollment(SID, SName, CourseID, CourseName, Faculty, Grade). Analyze FDs and apply 2NF decomposition.                  | BL3 – Apply      | 5     |                 |
| 4         | Given the airline booking case: Booking(FlightNo, PassID, PassName, Seat, DeptCity, ArrCity, Date). Normalize up to BCNF.                               | BL3 – Apply      | 5     |                 |
| 5         | Analyze an HR system schema for a multinational company. Identify all functional and transitive dependencies and normalize to 3NF.                      | BL2 – Understand | 5     |                 |

|    |                                                                                                                                        |                  |   |
|----|----------------------------------------------------------------------------------------------------------------------------------------|------------------|---|
| 6  | Case: Library(MemberID, MemberName, BookID, Title, Author, Genre, BorrowDate). Identify MVDs and decompose to 4NF.                     | BL3 – Apply      | 5 |
| 7  | Analyze a telecom billing schema and identify all candidate keys, primary keys, and functional dependencies before normalization.      | BL2 – Understand | 5 |
| 8  | A retail inventory system has Product(PID, PName, SupplierID, SupplierName, Category, Price, Warehouse). Apply complete normalization. | BL3 – Apply      | 5 |
| 9  | Study the given poorly normalized schema for a School ERP and propose a fully normalized design up to BCNF with justification.         | BL3 – Apply      | 5 |
| 10 | Given a movie streaming platform schema, identify join dependencies and decompose to 5NF with lossless-join verification.              | BL3 – Apply      | 5 |

---

**Code of Conduct:**

*I RASINENI HARIKA (192572140)(Name / Reg No) certify that this submission is my original work  
and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*



**Signature of the student:**

**RUBRICS FOR CALCULATION:**

**Rubric Reference (for self-check before submission)**

| Criteria                | Weightage | What Level-4 (Exemplary) looks like                                      |
|-------------------------|-----------|--------------------------------------------------------------------------|
| Understanding of Case   | 25        | Thorough understanding; all key issues (FDs, keys, anomalies) identified |
| Application of Concepts | 25        | Correct normal-form rules applied precisely (1NF→required NF)            |
| Analysis                | 20        | Clear reasoning for every FD and every decomposition step                |
| Solution Design         | 20        | Final schema is correct, lossless, and dependency-preserving             |

|         |    |                                                                     |
|---------|----|---------------------------------------------------------------------|
| Clarity | 10 | Well-structured answer: FDs → issue → decomposition → justification |
|---------|----|---------------------------------------------------------------------|

**Q1. Order(OrderID, CustID, CustName, ProdID, ProdName, Qty, Price) — Identify FDs and normalize to 3NF.**

**Answer:**

**Step 1: Functional Dependencies**

- OrderID → CustID
- CustID → CustName
- ProdID → ProdName, Price
- (OrderID, ProdID) → Qty

**Step 2: Key**

Primary key = (OrderID, ProdID). Since CustID, CustName, ProdName and Price depend only on part of the key, the table is not in 2NF.

**Step 3: 2NF — remove partial dependencies**

- Order(OrderID, CustID)
- Product(ProdID, ProdName, Price)
- OrderDetails(OrderID, ProdID, Qty)

**Step 4: 3NF — remove transitive dependency (CustID → CustName)**

- Customer(CustID, CustName)
- Order(OrderID, CustID)
- Product(ProdID, ProdName, Price)
- OrderDetails(OrderID, ProdID, Qty)

Result: All non-key attributes now depend only on the whole key of their own table — schema is in 3NF.

**Q2. Patient(PID, PName, DocID, DocName, Specialization, WardNo, WardName) — Identify anomalies and normalize to BCNF.**

**Answer:**

**Step 1: Functional Dependencies**

- PID → PName, DocID, WardNo
- DocID → DocName, Specialization
- WardNo → WardName

**Step 2: Anomalies in the un-normalized table**

- Update anomaly: changing a doctor's specialization must be repeated in every row of that doctor's patients.
- Insert anomaly: a new doctor cannot be added unless at least one patient is assigned.
- Delete anomaly: discharging the last patient of a doctor/ward deletes the doctor's and ward's details too.

**Step 3: BCNF decomposition**

- Patient(PID, PName, DocID, WardNo)
- Doctor(DocID, DocName, Specialization)
- Ward(WardNo, WardName)

Every determinant (PID, DocID, WardNo) is now a candidate key of its own relation, so all three tables satisfy BCNF.

**Q3. Enrollment(SID, SName, CourseID, CourseName, Faculty, Grade) — Analyze FDs and apply 2NF decomposition.**

**Answer:**

**Step 1: Functional Dependencies**

- $SID \rightarrow SName$
- $CourseID \rightarrow CourseName, Faculty$
- $(SID, CourseID) \rightarrow Grade$

**Step 2: Key and violation**

Primary key = (SID, CourseID). SName depends only on SID and CourseName/Faculty depend only on CourseID — both are partial dependencies, so the table violates 2NF.

**Step 3: 2NF decomposition**

- Student(SID, SName)
- Course(CourseID, CourseName, Faculty)
- Enrollment(SID, CourseID, Grade)

Every non-key attribute now depends on the whole primary key of its relation — the design satisfies 2NF.

**Q4. Booking(FlightNo, PassID, PassName, Seat, DeptCity, ArrCity, Date) — Normalize up to BCNF.**

**Answer:**

**Step 1: Functional Dependencies**

- $FlightNo \rightarrow DeptCity, ArrCity$
- $PassID \rightarrow PassName$
- $(FlightNo, Date, Seat) \rightarrow PassID$
- $(FlightNo, Date, PassID) \rightarrow Seat$

**Step 2: Candidate keys**

(FlightNo, Date, Seat) and (FlightNo, Date, PassID) are both candidate keys. FlightNo alone and PassID alone are determinants that are not superkeys, so the relation violates BCNF.

**Step 3: BCNF decomposition**

- Flight(FlightNo, DeptCity, ArrCity)
- Passenger(PassID, PassName)
- Booking(FlightNo, Date, Seat, PassID)

Every remaining determinant is a superkey of its table, so all three relations are in BCNF.

**Q5. HR system schema for a multinational company — Identify FDs/transitive dependencies and normalize to 3NF.**

**Answer:**

Assume Employee(EmpID, EmpName, DeptID, DeptName, DesigID, DesigTitle, CountryCode, CountryName).

**Step 1: Functional Dependencies**

- $\text{EmpID} \rightarrow \text{EmpName}, \text{DeptID}, \text{DesigID}, \text{CountryCode}$
- $\text{DeptID} \rightarrow \text{DeptName}$
- $\text{DesigID} \rightarrow \text{DesigTitle}$
- $\text{CountryCode} \rightarrow \text{CountryName}$

**Step 2: Transitive dependencies**

- $\text{EmpID} \rightarrow \text{DeptID} \rightarrow \text{DeptName}$
- $\text{EmpID} \rightarrow \text{DesigID} \rightarrow \text{DesigTitle}$
- $\text{EmpID} \rightarrow \text{CountryCode} \rightarrow \text{CountryName}$

**Step 3: 3NF decomposition**

- $\text{Employee}(\text{EmpID}, \text{EmpName}, \text{DeptID}, \text{DesigID}, \text{CountryCode})$
- $\text{Department}(\text{DeptID}, \text{DeptName})$
- $\text{Designation}(\text{DesigID}, \text{DesigTitle})$
- $\text{Country}(\text{CountryCode}, \text{CountryName})$

Removing the transitive dependencies eliminates duplicate department/designation/country data across employees — schema is in 3NF.

**Q6. Library(MemberID, MemberName, BookID, Title, Author, Genre, BorrowDate) — Identify MVDs and decompose to 4NF.****Answer:****Step 1: Multi-valued dependencies**

- A book can have several independent Authors and several independent Genres, so:  
 $\text{BookID} \twoheadrightarrow \text{Author}$
- $\text{BookID} \twoheadrightarrow \text{Genre}$  (Author and Genre are independent of each other, given BookID)

**Step 2: Why it violates 4NF**

Storing Author and Genre together in one row forces every author to be repeated with every genre for the same book — a classic 4NF anomaly caused by two independent multi-valued facts in one relation.

**Step 3: 4NF decomposition**

- $\text{Member}(\text{MemberID}, \text{MemberName})$
- $\text{Book}(\text{BookID}, \text{Title})$
- $\text{BookAuthor}(\text{BookID}, \text{Author})$
- $\text{BookGenre}(\text{BookID}, \text{Genre})$
- $\text{Borrow}(\text{MemberID}, \text{BookID}, \text{BorrowDate})$

Each relation now has at most one independent multi-valued fact, satisfying 4NF.

**Q7. Telecom billing schema — Identify all candidate keys, primary key, and FDs before normalization.****Answer:**

Assume  $\text{CallBill}(\text{CallID}, \text{CustID}, \text{CustName}, \text{PlanID}, \text{PlanName}, \text{CallDate}, \text{Duration}, \text{Amount})$ .

**Step 1: Functional Dependencies**

- $\text{CallID} \rightarrow \text{CustID}, \text{CallDate}, \text{Duration}, \text{Amount}$

- $\text{CustID} \rightarrow \text{CustName}, \text{PlanID}$
- $\text{PlanID} \rightarrow \text{PlanName}$

**Step 2: Candidate keys**

CallID is the only attribute that functionally determines every other attribute (directly or transitively), so CallID is the sole candidate key.

**Step 3: Primary key**

Primary Key = CallID.

**Step 4: Observation before normalization**

$\text{CustID} \rightarrow \text{CustName}, \text{PlanID}$  and  $\text{PlanID} \rightarrow \text{PlanName}$  are transitive dependencies on the key, so the table is currently only in 1NF and needs decomposition (Customer, Plan, Call) to reach 3NF.

**Q8. Product(PID, PName, SupplierID, SupplierName, Category, Price, Warehouse) — Apply complete normalization.**

**Answer:**

**Step 1: Functional Dependencies**

- $\text{PID} \rightarrow \text{PName}, \text{SupplierID}, \text{Category}, \text{Price}, \text{Warehouse}$
- $\text{SupplierID} \rightarrow \text{SupplierName}$

**Step 2: 1NF**

All attributes hold atomic values — condition already satisfied.

**Step 3: 2NF**

PID is a single-attribute primary key, so no partial dependency can exist — the table is already in 2NF.

**Step 4: 3NF / BCNF — remove transitive dependency**

- Product(PID, PName, SupplierID, Category, Price, Warehouse)
- Supplier(SupplierID, SupplierName)

PID is the key of Product and SupplierID is the key of Supplier — every determinant is a candidate key, so the design satisfies both 3NF and BCNF.

**Q9. School ERP — propose a fully normalized design up to BCNF with justification.**

**Answer:**

Assume the poorly normalized relation: Student(SID, SName, ClassID, ClassName, TeacherID, TeacherName).

**Step 1: Functional Dependencies**

- $\text{SID} \rightarrow \text{SName}, \text{ClassID}$
- $\text{ClassID} \rightarrow \text{ClassName}, \text{TeacherID}$
- $\text{TeacherID} \rightarrow \text{TeacherName}$

**Step 2: Problem**

ClassName and TeacherName are transitively dependent on SID ( $\text{SID} \rightarrow \text{ClassID} \rightarrow \text{ClassName}/\text{TeacherID} \rightarrow \text{TeacherName}$ ), which causes update/insert/delete anomalies.

**Step 3: BCNF decomposition**

- Student(SID, SName, ClassID)
- Class(ClassID, ClassName, TeacherID)

- Teacher(TeacherID, TeacherName)

#### **Justification**

SID, ClassID and TeacherID are each candidate keys of their own relation, so every determinant is a superkey — the design satisfies BCNF and removes all transitive redundancy.

#### **Q10. Movie streaming platform schema — identify join dependencies and decompose to 5NF with lossless-join verification.**

##### **Answer:**

Assume Streaming(Movie, Actor, Genre), where a movie's set of actors and set of genres are independent of each other, but which actor is associated with which genre only makes sense combined through the movie.

##### **Step 1: Join Dependency**

The relation satisfies JD(R1, R2, R3) where:

- R1(Movie, Actor)
- R2(Movie, Genre)
- R3(Actor, Genre)

This means Streaming can be losslessly reconstructed only by joining all three projections together — a three-way (not pairwise) dependency, which violates 4NF/5NF.

##### **Step 2: 5NF decomposition**

- MovieActor(Movie, Actor)
- MovieGenre(Movie, Genre)
- ActorGenre(Actor, Genre)

##### **Step 3: Lossless-join verification**

Natural-joining MovieActor with MovieGenre on Movie, then joining that result with ActorGenre on (Actor, Genre), reproduces exactly the original Streaming rows with no spurious tuples — confirming the decomposition is lossless and the schema is in 5NF.